Practical no : 1

Aim

To study and implement SQL table creation with constraints such as Primary Key, Foreign Key, NOT NULL, and perform SQL queries on banking database relations.

Objective

- To create relational database tables for a banking system.
- To apply integrity constraints like Primary Key, Foreign Key, and NOT NULL.
- To retrieve data using SQL queries with conditions and joins.
- To understand relationships between different tables in a database.

Theory

A Relational Database Management System (RDBMS) stores data in the form of tables. Relations between tables are maintained using keys and constraints.

Important Constraints

- **Primary Key (PK):** Uniquely identifies each record in a table.
- **Foreign Key (FK):** Maintains relationship between two tables.
- **NOT NULL:** Ensures that a column cannot contain NULL values.

Relations Used

1. **Account** – Stores account details.
2. **Branch** – Stores branch information.
3. **Customer** – Stores customer information.
4. **Depositor** – Relationship between customer and account.
5. **Loan** – Stores loan details.
6. **Borrower** – Relationship between customer and loan.

Conclusion

Thus, the banking database tables were successfully created using SQL with appropriate constraints such as Primary Key, Foreign Key, and NOT NULL. Various SQL queries were executed to retrieve required information from the database using selection, joins, filtering, ordering, and DISTINCT operations. This experiment helped in understanding relational database design and query execution in SQL.

Practical no 2

Aim

To create and manage banking database tables using SQL with appropriate constraints and perform queries using set operations, joins, aggregate functions, and grouping.

Objective

- To create relational tables with constraints such as Primary Key, Foreign Key, and NOT NULL.
- To understand relationships between banking entities.
- To perform SQL queries using JOIN, UNION, aggregate functions, and grouping.
- To retrieve meaningful information from the banking database.

Theory

In a Relational Database Management System (RDBMS), data is organized into tables related through keys.

Constraints Used

- **Primary Key:** Uniquely identifies each record in a table.
- **Foreign Key:** Maintains referential integrity between related tables.
- **NOT NULL:** Ensures mandatory fields are not left empty.

Relations

- **Branch:** Stores branch details.
- **Account:** Stores account information of customers.
- **Customer:** Contains customer details.
- **Depositor:** Represents relationship between customer and account.
- **Loan:** Stores loan details.
- **Borrower:** Represents relationship between customer and loan.

Conclusion

- Thus, the banking database tables were successfully created with appropriate integrity constraints. Various SQL queries were executed using JOIN, UNION, aggregate functions, and grouping to retrieve useful information from the database. This experiment improved understanding of relational database design, data relationships, and SQL query processing.

Practical no 3

Aim

To create banking database tables using SQL with appropriate constraints and perform SQL queries using aggregate functions, grouping, filtering, pattern matching, and deletion operations.

Objective

- To create relational database tables with integrity constraints.
- To understand the use of Primary Key, Foreign Key, and NOT NULL constraints.
- To perform SQL queries using aggregate functions like AVG(), COUNT(), and SUM().
- To implement filtering, grouping, deletion, and pattern matching operations in SQL.

Theory

A Relational Database Management System (RDBMS) organizes data into related tables. SQL is used to define, manipulate, and query data stored in these tables.

Constraints Used

- **Primary Key:** Ensures unique identification of each record.
- **Foreign Key:** Maintains relationship between tables.
- **NOT NULL:** Prevents NULL values in mandatory fields.

Aggregate Functions

- **AVG()** → Calculates average values.
- **COUNT()** → Counts records.
- **SUM()** → Calculates total values.

SQL Clauses Used

- **GROUP BY** → Groups rows having common values.
- **HAVING** → Filters grouped records.
- **LIKE** → Used for pattern matching.
- **DELETE** → Removes records from a table.

Conclusion

- Thus, the banking database tables were successfully created with proper constraints such as Primary Key, Foreign Key, and NOT NULL. Various SQL operations including aggregate functions, grouping, filtering, deletion, and pattern matching were performed successfully. This experiment enhanced understanding of relational database concepts and SQL query execution.

Practical no 4

Aim

To create customer, order, and product tables using SQL with suitable constraints and perform various SQL queries using pattern matching, filtering, views, and join operations.

Objective

- To create relational database tables with Primary Key, Foreign Key, and NOT NULL constraints.
- To insert records into tables.
- To perform SQL queries using LIKE, IN, JOIN, and VIEW operations.
- To understand different join operations in SQL.

Theory

A Relational Database Management System (RDBMS) stores data in tables related through keys.

Constraints Used

- **Primary Key:** Uniquely identifies each record in a table.
- **Foreign Key:** Establishes relationship between tables.
- **NOT NULL:** Ensures mandatory values are entered.

SQL Concepts Used

- **LIKE Operator:** Used for pattern matching.
- **IN Operator:** Used to match multiple values.
- **VIEW:** Virtual table created from query results.
- **JOIN Operations:** Used to combine rows from multiple tables.

Types of Joins

- **INNER JOIN:** Returns matching records from both tables.
- **LEFT JOIN:** Returns all records from left table and matching records from right table.
- **RIGHT JOIN:** Returns all records from right table and matching records from left table.
- **FULL OUTER JOIN:** Returns all matching and non-matching records.

Conclusion

Thus, the customer, order, and product tables were successfully created with suitable constraints such as Primary Key, Foreign Key, and NOT NULL. Records were inserted successfully, and various SQL queries using LIKE, IN, VIEW, and JOIN operations were executed. This experiment helped in understanding relational database design and SQL query processing with different join operations.

Practical no : 5

Aim

To create Employee, Works, Company, and Manages tables using SQL with appropriate constraints and perform SQL queries for data retrieval, sorting, updating, and altering tables.

Objective

- To create relational database tables with Primary Key, Foreign Key, and NOT NULL constraints.
- To understand relationships among employee, company, and management tables.
- To perform SQL operations such as SELECT, UPDATE, ORDER BY, and ALTER TABLE.
- To retrieve and manipulate employee and company information using SQL queries.

Theory

A Relational Database Management System (RDBMS) stores data in related tables. SQL is used to define database structures and manipulate data efficiently.

Constraints Used

- **Primary Key:** Uniquely identifies each record in a table.
- **Foreign Key:** Maintains relationships between tables.
- **NOT NULL:** Ensures that mandatory fields contain values.

SQL Operations Used

- **SELECT:** Retrieves records from tables.
- **ORDER BY:** Sorts data in ascending or descending order.
- **UPDATE:** Modifies existing records.
- **ALTER TABLE:** Adds or modifies columns in existing tables.

Relations

- **Employee:** Stores employee details.
- **Works:** Stores employee company and salary information.
- **Company:** Stores company details.
- **Manages:** Stores manager-employee relationship.

Conclusion

- Thus, the Employee, Works, Company, and Manages tables were successfully created with suitable constraints such as Primary Key, Foreign Key, and NOT NULL. Various SQL operations like SELECT, UPDATE, ORDER BY, JOIN, and ALTER TABLE were performed successfully. This experiment improved understanding of relational database management and SQL query execution for employee-company relationships.

Practical no 6

Aim

To create Companies and Orders tables using SQL with appropriate constraints and perform different join operations, UNION operation, and VIEW operations.

Objective

- To create relational database tables with suitable constraints.

- To understand and implement INNER JOIN, LEFT OUTER JOIN, and RIGHT OUTER JOIN operations.
- To perform UNION operations on tables.
- To create and manipulate SQL Views using INSERT, UPDATE, and DELETE operations.

Theory

A Relational Database Management System (RDBMS) stores related data in tables. SQL joins are used to combine records from multiple tables based on common attributes.

Constraints Used

- **Primary Key:** Uniquely identifies each record.
- **Foreign Key:** Creates relationship between tables.
- **NOT NULL:** Ensures mandatory fields contain values.

SQL Operations Used

- **INNER JOIN:** Returns only matching rows from both tables.
- **LEFT OUTER JOIN:** Returns all rows from left table and matching rows from right table.
- **RIGHT OUTER JOIN:** Returns all rows from right table and matching rows from left table.
- **UNION:** Combines results from two SELECT statements without duplicates.
- **VIEW:** Virtual table based on SQL query results.

Conclusion

Thus, the Companies and Orders tables were successfully created with suitable constraints such as Primary Key, Foreign Key, and NOT NULL. Different join operations including INNER JOIN, LEFT OUTER JOIN, and RIGHT OUTER JOIN were performed successfully. Views were also created and manipulated using INSERT, UPDATE, and DELETE operations. This experiment enhanced understanding of SQL joins, set operations, and view management in relational databases.

Practical no 7

Aim

To create CUSTOMER, ITEMS, and PURCHASE tables with suitable constraints and perform SQL queries using filtering, sorting, aggregate functions, updating, joins, and views.

Objective

- To create relational database tables with Primary Key, Foreign Key, and NOT NULL constraints.
- To insert records into the tables.
- To perform SQL queries using WHERE, ORDER BY, aggregate functions, and JOIN operations.
- To create and use SQL VIEW for displaying selected data.

Theory

A Relational Database Management System (RDBMS) stores data in related tables. SQL is used to create, manipulate, and retrieve data from these tables.

Constraints Used

- **Primary Key:** Uniquely identifies each record in a table.
- **Foreign Key:** Establishes relationship between tables.
- **NOT NULL:** Prevents NULL values in mandatory fields.

SQL Concepts Used

- **BETWEEN:** Filters values within a specified range.
- **UPDATE:** Modifies existing records.
- **MAX():** Finds maximum value.
- **ORDER BY:** Sorts records.
- **COUNT():** Counts number of records.
- **JOIN:** Combines data from multiple tables.
- **VIEW:** Creates a virtual table from query results.

Conclusion

- Thus, the CUSTOMERS, ITEMS, and PURCHASE tables were successfully created with suitable constraints such as Primary Key, Foreign Key, and NOT NULL. Records were inserted successfully, and various SQL queries using filtering, aggregate functions, sorting, joins, and views were executed. This experiment improved understanding of relational database operations and SQL query execution.

Practical no 8

Aim

To design and develop MongoDB queries using CRUD operations on an Employee collection with embedded documents and arrays.

Objective

- To create an Employee collection in MongoDB.
- To insert documents with embedded documents and arrays.
- To perform CRUD operations such as Insert, Find, Update, and Conditional Queries.
- To understand MongoDB document structure and query syntax.

Theory

MongoDB is a NoSQL database that stores data in the form of JSON-like documents. It supports flexible schema design using embedded documents and arrays.

MongoDB Concepts Used

- **Collection:** Group of MongoDB documents.
- **Document:** Record stored in BSON format.
- **Embedded Document:** Document stored inside another document.
- **Array:** Stores multiple values in a single field.

CRUD Operations

- **Create:** Insert documents.
- **Read:** Retrieve documents using `find()`.

- **Update:** Modify existing documents using `updateOne()` or `updateMany()`.
- **Delete:** Remove documents.

MongoDB Operators Used

- `$gt` → Greater than
- `$ne` → Not equal
- `$inc` → Increment value
- `upsert:true` → Insert document if not found

Conclusion

- Thus, the Employee collection was successfully created in MongoDB with embedded documents and arrays. CRUD operations such as insert, find, and update were performed successfully using MongoDB queries. This experiment helped in understanding MongoDB document structure, embedded documents, arrays, and query operators.

Practical no 9

Aim

To design and develop MongoDB queries using CRUD operations on an Employee collection containing embedded documents and arrays.

Objective

- To create an Employee collection in MongoDB.
- To insert documents with embedded documents and arrays.
- To perform CRUD operations such as insert, find, update, and projection.
- To understand MongoDB query operators and document structure.

Theory

MongoDB is a NoSQL database that stores data in the form of BSON documents. It supports flexible schema design using embedded documents and arrays.

MongoDB Concepts Used

- **Collection:** Group of MongoDB documents.

- **Document:** Record stored in BSON format.
- **Embedded Document:** Document stored inside another document.
- **Array:** Field containing multiple values.

CRUD Operations

- **Create:** Insert documents using `insertMany()`.
- **Read:** Retrieve documents using `find()`.
- **Update:** Modify records using `updateOne()` or `updateMany()`.

MongoDB Operators Used

- `$gt` → Greater than
- `$lt` → Less than
- `$or` → Logical OR
- `$ne` → Not equal
- `$inc` → Increment/Decrement values
- `$setOnInsert` → Sets values only during insertion
- `upsert:true` → Inserts document if not found

Conclusion

- Thus, the Employee collection was successfully created in MongoDB using embedded documents and arrays. CRUD operations such as insert, find, update, and projection were performed successfully using MongoDB queries. This experiment enhanced understanding of MongoDB document structure, query operators, embedded documents, and array handling.

Practical no 10

Aim

To design a MongoDB database using the Employee collection and perform MapReduce operations to calculate aggregated results such as total salary, average salary, and count-based queries.

Objective

- To create an Employee collection with embedded documents and arrays.
- To understand MapReduce operations in MongoDB.
- To perform aggregation such as SUM, COUNT, and AVERAGE using MapReduce.
- To filter and process data based on conditions like company name, city, and age.

Theory

MongoDB MapReduce is a data processing technique used for large-scale aggregation.

MapReduce Phases

- **Map Function:** Processes each document and emits key-value pairs.
- **Reduce Function:** Aggregates values for the same key.
- **Finalize Function (optional):** Performs final calculations on reduced output.

Concepts Used

- Embedded Documents (Name, Address)
- Arrays (Expertise, Address)
- Conditional filtering
- Aggregation logic using JavaScript functions

Employee Collection Structure

Each employee document contains:

- Name (FName, LName)
- Company_name
- Salary
- Designation
- Age
- Expertise (Array)
- DOB
- Email_id
- Contact
- Address (PAddr, LAddr)

Conclusion

- Thus, the Employee collection was successfully created in MongoDB and MapReduce operations were performed. Aggregations such as total salary, average salary, and count based on conditions like company name, city, and age were successfully implemented. This experiment helped in understanding MapReduce programming model, data aggregation, and processing large datasets in MongoDB.

Practical no 11

Aim

To write a PL/SQL block that calculates the area of a circle for radius values from 5 to 9 and stores the results in a database table.

Objective

- To understand PL/SQL block structure.
- To use loops in PL/SQL.
- To perform mathematical calculations in Oracle PL/SQL.
- To insert computed values into a database table.

Theory

PL/SQL (Procedural Language/SQL) is Oracle's extension of SQL that allows procedural programming features such as loops, conditions, and variables.

Key Concepts Used

- **DECLARE block:** Used to define variables.
- **BEGIN block:** Contains executable statements.
- **LOOP / FOR loop:** Used for iteration.
- **INSERT statement:** Used to store results in a table.
- **COMMIT:** Saves changes permanently.

Formula Used

Area of circle = $\pi \times r^2$
($\pi \approx 3.14$)

Conclusion

Thus, the PL/SQL block was successfully executed to calculate the area of a circle for radius values ranging from 5 to 9. The computed results were stored in the `areas` table, demonstrating the use of loops, variables, and SQL operations inside PL/SQL.

Practical no 12

Aim

To write an unnamed PL/SQL block to calculate fine based on book return delay, update borrower status, and store fine details into the Fine table.

Objective

- To use PL/SQL block without procedure name (anonymous block).
- To accept input values using variables.
- To calculate number of days between issue and return.
- To apply conditional logic for fine calculation.
- To update table status and insert fine records.

Theory

PL/SQL (Procedural Language/SQL) is used in Oracle to execute procedural logic along with SQL statements.

Key Concepts Used

- **Anonymous PL/SQL Block:** A block without a name.
- **Variables:** Used to store input and computed values.
- **IF-THEN-ELSE:** Used for decision making.
- **Date Arithmetic:** Used to calculate difference between dates.
- **INSERT & UPDATE:** Used to modify database tables.

Schema Used

- **Borrower(Rollin, Name, DateofIssue, NameofBook, Status)**
- **Fine(Roll_no, Date, Amt)**

Working Explanation

1. User enters Roll Number and Book Name.
2. System fetches issue date from Borrower table.
3. Calculates number of days using SYSDATE.
4. Applies fine rules:
    - 15–30 days → Rs. 5 per day
    - 30 days → Rs. 50 per day

    - <15 days → No fine
5. Updates status from 'I' (Issued) to 'R' (Returned).
6. Inserts fine details into Fine table if applicable.

Conclusion

Thus, the unnamed PL/SQL block was successfully written to calculate library fines based on book return delay. It also updated borrower status and inserted fine records into the Fine table, demonstrating the use of conditional statements, exception handling, and DML operations in PL/SQL

Practical no 13

Aim

To write a PL/SQL block using a cursor to merge data from `N_RollCall` table into `O_RollCall` table while avoiding duplicate records.

Objective

- To understand the use of explicit cursors in PL/SQL.
- To fetch and process multiple rows from a table.
- To perform conditional insertion (merge-like operation).
- To avoid duplicate records during data transfer.

Theory

A **cursor** in PL/SQL is a pointer to the result set of a SQL query. It allows row-by-row processing.

Types of Cursors

- **Implicit Cursor**: Automatically created by Oracle.
- **Explicit Cursor**: Defined by the programmer for multi-row processing.

Concept Used in This Program

- Fetch data from `N_RollCall` using cursor.
- Check if record already exists in `O_RollCall`.
- Insert only non-duplicate records.

Working Explanation

1. Cursor `c1` fetches all records from `N_RollCall`.
2. Each record is processed one by one.
3. For every record, a check is performed in `O_RollCall`.
4. If the record does not exist, it is inserted.
5. If it already exists, it is skipped.
6. Finally, changes are committed.

Conclusion

Thus, a PL/SQL block using an explicit cursor was successfully written to merge data from `N_RollCall` into `O_RollCall` while avoiding duplicate entries. This program demonstrates cursor handling, row-by-row processing, and conditional insertion in PL/SQL.

Practical no 14

Aim

To write a PL/SQL block to check student attendance, update status in the `Student` table, and handle exceptions based on attendance criteria.

Objective

- To use PL/SQL for conditional processing.
- To accept user input using substitution variables.
- To update database records based on conditions.
- To handle exceptions such as missing records.
- To practice IF-ELSE logic in PL/SQL.

Theory

PL/SQL (Procedural Language/SQL) allows procedural programming in Oracle databases.

Key Concepts Used

- **Anonymous Block:** A PL/SQL block without a name.
- **Exception Handling:** Used to manage runtime errors.
- **Conditional Statements:** IF–THEN–ELSE logic.
- **DML Operations:** UPDATE statements.

Logic Used

- If attendance < 75% → "Term not granted" → Status = "Detained"
- If attendance ≥ 75% → "Term granted" → Status = "Not Detained"

Working Explanation

1. User enters Roll Number.
2. Attendance is fetched from `Student` table.
3. Condition is checked:
    - If attendance < 75 → student is detained.
    - Else → term is granted.
4. Status is updated accordingly in the table.
5. Exceptions are handled:
    - `NO_DATA_FOUND` if roll number does not exist.
    - `OTHERS` for any unexpected errors.

Conclusion

Thus, a PL/SQL block was successfully written to check student attendance, update status in the Student table, and handle exceptions. The program demonstrates conditional logic, database update operations, and exception handling in PL/SQL.

Practical no 15

Aim

To write a PL/SQL block that increases the salary of employees by 10% whose salary is less than the average salary of the organization and logs the updated records into an `increment_salary` table.

Objective

- To use PL/SQL for conditional updates.
- To calculate aggregate values (average salary).
- To update employee salary using arithmetic operations.
- To maintain an audit trail using another table.
- To practice exception handling and DML operations in PL/SQL.

Theory

PL/SQL is Oracle's procedural extension of SQL that allows writing logic using loops, conditions, and variables.

Key Concepts Used

- **Aggregate Function (AVG):** Used to calculate average salary.
- **Conditional Update:** Salary updated only if condition is satisfied.
- **Audit Table:** Stores updated employee details.
- **DML Operations:** UPDATE and INSERT statements.

Logic Used

- Find average salary of all employees.
- If employee salary < average salary:
    - Increase salary by 10%
    - Insert record into `increment_salary`

Working Explanation

1. Average salary of all employees is calculated.
2. Employees earning less than average are identified.
3. Their salary is increased by 10%.
4. Updated records are inserted into `increment_salary` table.
5. Changes are committed to the database.
6. Exceptions are handled for missing data or runtime errors.

Conclusion

Thus, a PL/SQL block was successfully written to increase the salary of eligible employees and maintain a record of updates in the `increment_salary` table. This demonstrates the use of aggregate functions, conditional updates, and auditing using PL/SQL.

Practical no 16

Aim

To create a stored function named **Age_calc** in PL/SQL that calculates a person's age from Date of Birth and returns age in years, while months and days are returned using OUT parameters.

Objective

- To understand stored functions in PL/SQL.
- To use IN and OUT parameters in a function/procedure.
- To perform date calculations using Oracle date functions.
- To return multiple values using a combination of RETURN and OUT parameters.

Theory

A **stored function** in PL/SQL is a named block that returns a value using the `RETURN` statement.

Key Concepts Used

- **IN Parameter:** Input value (Date of Birth).
- **OUT Parameters:** Used to return multiple values (months and days).
- **SYSDATE:** Current system date.
- **MONTHS_BETWEEN():** Calculates difference in months between two dates.
- **Date arithmetic:** Used for calculating days and months.

Logic

- Calculate total months between DOB and current date.
- Extract:
  - Years = total_months / 12
  - Remaining months = MOD(total_months, 12)
- Calculate remaining days separately.
- Return years using RETURN statement.
- Return months and days using OUT parameters.

Conclusion

Thus, a stored function named **Age_calc** was successfully created in PL/SQL to calculate a person's age in years, months, and days. The function demonstrates the use of IN and OUT parameters, date functions, and return values in PL/SQL programming.

Practical no 17

Aim

To create **Row Level BEFORE and AFTER Triggers** on a Library table to track updates and deletions, and store old records in a **Library_Audit** table for auditing purposes.

Objective

- To understand Row Level Triggers in PL/SQL.
- To use BEFORE and AFTER triggers on UPDATE and DELETE operations.
- To maintain an audit trail of modified and deleted records.
- To store old values in an audit table for tracking changes.

Theory

A **trigger** is a stored PL/SQL block that automatically executes when a specific event occurs on a table.

Types of Triggers Used

- **BEFORE Trigger:** Executes before the DML operation.
- **AFTER Trigger:** Executes after the DML operation.
- **Row Level Trigger:** Fires once for each row affected.

Purpose of Audit Trail

Audit tables are used to:

- Track data modifications.
- Store historical data.
- Maintain accountability of changes.

Conclusion

- Thus, Row Level BEFORE and AFTER triggers were successfully created on the Library table. The system effectively tracks update and delete operations by storing old records in the Library_Audit table, demonstrating the use of triggers for audit and data integrity management in PL/SQL.

Practical no 18

Aim

To create a **row-level trigger** on the `CUSTOMERS` table that fires on **INSERT, UPDATE, and DELETE** operations and displays the salary difference between old and new values.

Objective

- To understand row-level triggers in PL/SQL.
- To handle multiple DML operations (INSERT, UPDATE, DELETE) in a single trigger.
- To use `:OLD` and `:NEW` values.
- To calculate and display salary differences during data modifications.

Theory

A **trigger** is a PL/SQL block that automatically executes when a specified event occurs on a table.

Row-Level Trigger

- Fires once for each row affected.
- Uses `FOR EACH ROW`.

Pseudorecords

- `:OLD` → holds previous values (before change)
- `:NEW` → holds new values (after change)

Events Covered

- INSERT → new salary available
- UPDATE → salary changed (difference calculated)
- DELETE → salary removed

Working Explanation

1. Trigger fires automatically on INSERT, UPDATE, DELETE.
2. Uses `INSERTING`, `UPDATING`, and `DELETING` conditions.
3. For UPDATE:
    - Computes salary difference using `:NEW.Salary - :OLD.Salary`.
4. Displays output using `DBMS_OUTPUT.PUT_LINE`.

Conclusion

Thus, a row-level trigger was successfully created on the CUSTOMERS table. The trigger handles INSERT, UPDATE, and DELETE operations and displays salary differences between old and new values, demonstrating the use of row-level triggers and pseudorecords in PL/SQL.

Practical no 19

Aim

To create an **AFTER trigger** on the `Emp` table that fires on **INSERT, UPDATE, and DELETE** operations, validates salary conditions, and stores valid salary changes in a `Tracking` table.

Objective

- To understand AFTER row-level triggers in PL/SQL.
- To apply business rules on INSERT and UPDATE operations.
- To restrict or handle salary values based on conditions.
- To store valid changes in a tracking/audit table.

Theory

A **trigger** is a PL/SQL block that executes automatically in response to events like INSERT, UPDATE, or DELETE.

AFTER Trigger

- Executes after the triggering event.
- Used for auditing and logging purposes.

Row-Level Trigger

- Fires once for each affected row using `FOR EACH ROW`.

Business Rule

- Salary must be **≥ 50,000**
- If salary is less than 50,000 during INSERT or UPDATE, trigger should activate and handle it.
- Valid salary records are stored in `Tracking` table.

Conclusion

- Thus, an AFTER row-level trigger was successfully created on the Emp table. It enforces salary validation rules for INSERT and UPDATE operations and stores valid employee salary details in the Tracking table, demonstrating the use of business logic enforcement and auditing in PL/SQL triggers.